Examen du cours de Programmation et Algorithmique II

1ère Session, Juin 2023

Partie 1

NOM : ..... PRENOM : ..... SECTION : .....

Consignes à lire impérativement !

L'examen est composé de **2 parties**. Chaque partie dure **2 heures**. Il vous est demandé de respecter les consignes suivantes.

- Commencez par écrire vos **nom**, **prénom** et **section** (math, info, ...) sur chaque feuille, y compris les feuilles de brouillon.
- Laissez vos calculatrice, téléphone portable et notes de cours dans votre sac. Leur usage n'est **pas autorisé**. Pensez à éteindre votre téléphone portable !
- Faites attention à la clarté et à l'organisation de vos réponses. Respectez les règles grammaticales et orthographiques.
- Utilisez pour vos réponses les **cadres** prévus à cet effet. Si davantage d'espace est nécessaire, utilisez le dos de la feuille ou une feuille supplémentaire et indiquez clairement où se situe le restant de la réponse.
- Vous devez terminer cette partie de l'examen avant de pouvoir sortir de la salle (pour aller à la toilette par exemple).
- Toutes les feuilles (énoncé et brouillon) doivent être remises en fin d'examen.
- Vérifiez que vous avez répondu à toutes les questions (il y a **2 questions** dans cette partie).

Question 1 – Echauffement (/4)

Q1a – Quelle sera la valeur affichée par le programme ci-dessous ?

.....

```
class A {
    public int x = 3;
    public static int getX() {
        return x;
    }
}
class B extends A {
    public int x = 5;
    public static int getX() {
        return x;
    }
}
A ref = new B();
System.out.println(ref.getX());
```

Examen du cours de Programmation et Algorithmique II

1ère Session, Juin 2023

Partie 1

NOM : ..... PRENOM : ..... SECTION : .....

Q1b – Donnez une instruction permettant d'appeler la méthode `doSthg` de la classe `A` ?

.....

```
class A {
    public void doSthg() {
        System.out.println("A.doSthg");
    }
}
class B extends A {
    @Override
    public void doSthg() {
        System.out.println("B.doSthg");
    }
}
A ref = new B();
```

Q1c – Remplacez les commentaires dans le code suivant de sorte que la somme affichée vaille 23. La boucle doit effectuer 3 itérations et aucun des nombres du tableau ne peut être nul.

- ligne 1 : new Integer[]{-30, 6, 1 } .....
- ligne 3 : Integer d : tab .....

```
1  Integer [] tab = /* ... */ ;
2  int somme = 0;
3  for ( /* ... */ )
4      somme -= d;
5  System.out.println(somme);
```

Q1d – Quel sera le résultat affiché par le programme ci-dessous ?

15

.....

.....

```
int a = 60;
int b = 105;
while (b != 0) {
    int r = a % b;
    a = b;
    b = r;
}
System.out.println(a);
```

r = 60
a=105
b=60
r=45
a=60
b=45
r=15
a=45
b=15
r=0
a=15
b=0
print 15

Examen du cours de Programmation et Algorithmique II

1ère Session, Juin 2023

Partie 1

NOM : ..... PRENOM : ..... SECTION : .....

Q1e – Quel sera le résultat affiché par le programme ci-dessous ? (cochez la case correspondante)

- ☐ Le programme affiche « `zoouuu` ».
- ☐ Le programme affiche « `blam` ».
- ☒ Le programme n'affiche rien.
- ☐ Aucune des solutions ci-dessus.

```
int t = -7;
if (t >= 0)
    if (t == 0)
        System.out.println("zoouuu");
else
    System.out.println("blam");
```

Q1f – Quel sera l'effet du code ci-dessous où `E` est un paramètre de type ? (cochez la case correspondante)

- ☐ Le programme crée un tableau de 100 éléments.
- ☐ Le programme cause une erreur lors de l'exécution.
- ☒ Le programme ne compile pas.

```
E[] tab = new E[100];
```

Q1g – Quel sera le résultat affiché par le programme ci-dessous ? (cochez la case correspondante)

- ☐ Le programme affiche « `true` ».
- ☒ Le programme affiche « `false` ».
- ☐ Le programme ne compile pas.
- ☐ Le programme génère une exception.
- ☐ On ne peut pas savoir avec certitude.

```
String s1 = "Hello";
String s2 = "Hell";
s2 += 'o';
System.out.println(s1 == s2);
```

Examen du cours de Programmation et Algorithmique II
1ère Session, Juin 2023
Partie 1

NOM : ..... PRENOM : ..... SECTION : .....

Q1h – Quel modificateur devez-vous ajouter à la définition de la classe `FonctionSinus` ci-dessous ?

abstract

```
public interface Fonction {
    double eval(double x, double y);
}
public /* ... */ class FonctionSinus implements Fonction{
    double eval(double x) {
        return Math.sin(x);
    }
}
```

Question 2 – Suppression dans une liste chaînée triée (/6)

L'objectif de cette question est de fournir une méthode `remove` permettant la suppression d'un élément dans une liste **doublement** chaînée **triée**. Une liste chaînée repose sur la classe `LinkedList` partiellement illustrée ci-dessous. La classe interne `Node` définit la structure de chaque maillon de la liste chaînée. A cet effet, `Node` comprend une variable `data` référençant le contenu d'un maillon et les variables `prev` et `next` qui référencent respectivement les maillons précédents et suivants. Dans la classe `LinkedList`, les variables `head` et `tail` référencent respectivement les premiers et derniers maillons de la chaîne.

```
public class LinkedList< /* à compléter ( Q2a ) */ >
{
  private Node head, tail; /* tête et fin de liste */

  private class Node { /* Représentation d'un maillon */
    public final E data;
    public Node prev, next; /* maillons précédent et suivant */

    public Node(E data, Node prev, Node next) {
      this.data= data;
      this.prev = prev;
      this.next = next;
    }
  }

  /* Supprime l'élément e de la liste.
     Retourne true si l'élément existait, false sinon */
  public boolean remove(E e) { ... }                    /* à implémenter ( Q2b ) */ }

  /* Les autres méthodes ne sont pas montrées */

}
```

Il vous est demandé de fournir l'implémentation de la méthode `remove(E` e`)` qui supprime l'élément e de la liste chaînée. Si l'élément a été trouvé, la méthode retourne `true` sinon elle retourne `false`. Il est nécessaire de tirer parti

du fait que la liste est triée. Note : on considère que la liste chaînée ne peut contenir deux fois le même élément. Lorsque l'on veut supprimer un élément de la liste, on sait donc qu'il ne peut s'y trouver qu'une fois au plus. Il vous est également demandé de compléter la définition du paramètre de type dans la déclaration de la classe `LinkedList`.

Q2a – Définition du paramètre de type de la classe `LinkedList`

```
E extends Comparable<E>
```

Q2b – Implémentation de la méthode `remove` de la classe `LinkedList`

```
Node temp = head;

while (temp != null && !temp.data.equals(e)) {
    temp = temp.next;
}

if (temp == null) {
    return false;
}

if (temp.prev != null)
    temp.prev.next = temp.next;
else
    head = temp.next;

if (temp.next != null)
    temp.next.prev = temp.prev;
else
    tail = temp.prev;

return true;
```

NOM : . . . . . PRENOM : . . . . . SECTION : . . . . . .

Q2b – Implémentation de la méthode `remove` de la classe `LinkedList` (suite)

Examen du cours de Programmation et Algorithmique II

1ère Session, Juin 2023

Partie 2

NOM : ..... PRENOM : ..... SECTION : .....

Consignes à lire impérativement !

L'examen est composé de **2 parties**. Chaque partie dure **2 heures**. Il vous est demandé de respecter les consignes suivantes.

- Commencez par écrire vos **nom**, **prénom** et **section** (math, info, ...) sur chaque feuille, y compris les feuilles de brouillon.
- Laissez vos calculatrice, téléphone portable et notes de cours dans votre sac. Leur usage n'est **pas autorisé**. Pensez à éteindre votre téléphone portable !
- Faites attention à la clarté et à l'organisation de vos réponses. Respectez les règles grammaticales et orthographiques.
- Utilisez pour vos réponses les **cadres** prévus à cet effet. Si davantage d'espace est nécessaire, utilisez le dos de la feuille ou une feuille supplémentaire et indiquez clairement où se situe le restant de la réponse.
- Vous devez terminer cette partie de l'examen avant de pouvoir sortir de la salle (pour aller à la toilette par exemple).
- Toutes les feuilles (énoncé et brouillon) doivent être remises en fin d'examen.
- Vérifiez que vous avez répondu à toutes les questions (il y a **2 questions** dans cette partie).

Question 1 – Multiplication de matrices (/5)

L'objectif de cette question est l'implémentation d'un algorithme qui multiplie une matrice à deux dimensions par une autre. Vous disposez à cet effet d'une classe `Matrix` qui modélise une matrice $m \times n$ de nombres réels.

```
public class Matrix {
    public final double [][] content;

    public Matrix(int m, int n, double...content) {
        if (n * m != content.length)
            throw new IllegalArgumentException("nombre incorrect de valeurs");
        this.content = new double [][];
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                this.content[i][j] = content[i*n + j];
    }

    private Matrix(double [][] content) {
        this.content = content;
    }

    /* ... Méthode de classe multiply à fournir ( Q1a ) */
    /* ... Méthode d'instance multiply à fournir ( Q1b ) */
    /* ... Méthode d'instance equals à fournir ( Q1c ) */

}
```

Examen du cours de Programmation et Algorithmique II

1ère Session, Juin 2023

Partie 2

NOM : ..... PRENOM : ..... SECTION : .....

La classe `Matrix` représente une matrice à l'aide d'un tableau à deux dimensions nommé `content`. La première dimension du tableau correspond aux lignes de la matrice tandis que la seconde dimension correspond aux colonnes. La classe `Matrix` est dotée de deux constructeurs. Le premier est public et prend d'abord le nombre de lignes (`m`) et de colonnes (`n`) de la matrice. Vient ensuite un nombre d'arguments variables définissant les éléments de la matrice. Le nombre de valeurs doit correspondre à la taille de la matrice, $m \times n$. Le second constructeur est privé et permet de créer une instance de `Matrix` à partir d'un tableau à deux dimensions contenant les valeurs des éléments de la matrice.

Il vous est demandé d'ajouter à la classe `Matrix` la possibilité d'effectuer la multiplication d'une matrice par une autre. Le résultat de la multiplication est une nouvelle instance de `Matrix`. Vous devez fournir deux méthodes appelées `multiply` pour effectuer la multiplication. La première sera une méthode de classe tandis que la seconde sera une méthode d'instance. La seconde méthode doit utiliser la première pour effectuer la multiplication (inutile d'implémenter deux fois le même algorithme). Vous devez également re-définir la méthode `equals` héritée de la classe `Object`.

Q1a – Implémentation de la méthode de classe `multiply`

Examen du cours de Programmation et Algorithmique II

1ère Session, Juin 2023

Partie 2

NOM : ..... PRENOM : ..... SECTION : .....

Q1b – Implémentation de la méthode d'instance `multiply`

Q1c – Re-définition de la méthode `equals`

Le programme suivant illustre comment deux matrices A et B peuvent être modélisées par deux instances de `Matrix`. Le produit $A \times B$ est ensuite calculé.

```
Matrix A = new Matrix(2, 2, 1, 2, 3, 4);
Matrix B = new Matrix(2, 1, 5, 7);
Matrix C = /* ... appel de la méthode de classe multiply ( Q1d ) ... */;
Matrix D = /* ... appel de la méthode d'instance multiply ( Q1e ) ... */;
```

On vous demande également de compléter le programme d'exemple pour effectuer la multiplication $A \times B$ en invoquant la méthode de classe pour produire la matrice C et en invoquant la méthode d'instance pour produire la matrice D. Notez que C et D devraient être des instances différentes ayant le même contenu.

Q1d – Appel de la méthode de classe `multiply` pour calculer $A \times B$

Q1e – Appel de la méthode d'instance `multiply` pour calculer $A \times B$

Examen du cours de Programmation et Algorithmique II
1ère Session, Juin 2023
Partie 2

NOM : ..... PRENOM : ..... SECTION : .....

Question 2 – Table de Hachage (/5)

L'objectif de cette question est l'implémentation d'une table de hachage et de son itérateur. La table de hachage est implémentée à l'aide de la classe `HashMap` partiellement illustrée ci-dessous. La classe interne `HashEntry` représente une paire (clé, valeur). La variable `tab` référence le tableau de listes de paires (clé,valeur). Les listes sont implémentées à l'aide de la classe `LinkedList` de la bibliothèque standard. La méthode `put` permet d'ajouter l'association entre une clé et une valeur, tandis que la méthode `get` permet de récupérer la valeur associée à une clé. La méthode `iterator` permet de récupérer un itérateur sur la table de hachage, conformément à l'interface `Iterable`.

L'itérateur est implémenté à l'aide de la classe interne `HashMapIterator`. Elle fournit l'implémentation des deux méthodes prescrites par l'interface `Iterator`. La méthode `hasNext` permet de tester s'il existe un prochain élément à récupérer, tandis que la méthode `next` permet de récupérer cet élément et de faire avancer l'itérateur.

```
public class HashMap<K, V> implements Iterable<V> {

  private static class HashEntry {
    public K key;
    public V value;
    public HashEntry(K key, V value) {
      this.key = key;
      this.value = value;
    }
  }

  private List<HashEntry<K,V>> [] tab = ...

  public void put(K key, V value) { ... }                    /* à fournir ( Q2a ) */
  public V get(K key) { ... }
  public Iterator<V> iterator() { ... }                      /* à fournir ( Q2b ) */

  private class HashMapIterator implements Iterator<V> {
    ...                                                      /* à fournir ( Q2c ) */
  }

}
```

Il vous est demandé de fournir l'implémentation des éléments suivants :

1. Méthode `put` de la classe `HashMap` : ajoute une paire (clé, valeur) ou remplace une paire existante.
2. Méthode `iterator` de la classe `HashMap` : retourne un itérateur pour toutes les valeurs de la table de hachage.
3. Classe interne `HashMapIterator`. Elle doit contenir les méthodes `hasNext` et `next` ainsi que tout ce que vous jugez nécessaire, à savoir variables, autres méthodes et constructeur. Attention, il est possible que plusieurs paires contiennent la même valeur auquel cas cette valeur sera retournée plusieurs fois par l'itérateur. L'itérateur doit fonctionner directement sur la table de hachage et **ne peut en aucun cas** utiliser une structure de données intermédiaire telle que, par exemple, une liste. De plus, la méthode `hasNext` **ne doit aucunement faire avancer** l'itérateur : elle peut en effet être appelée plusieurs fois avant de récupérer le prochain élément avec `next`.

Examen du cours de Programmation et Algorithmique II
1ère Session, Juin 2023
Partie 2

NOM : . . . . . PRENOM : . . . . . SECTION : . . . . . .

Q2a – Implémentation de la méthode `put` de la classe `HashMap`

Q2b – Implémentation de la méthode `iterator` de la classe `HashMap`

Examen du cours de Programmation et Algorithmique II
1ère Session, Juin 2023
Partie 2

NOM : . . . . . PRENOM : . . . . . SECTION : . . . . . .

Q2c – Implémentation de la classs interne `HashMapIterator`